

COMPUTACIÓN II - Práctica 5:

(1a) - Escribir un programa que cree un array unidimensional dinámico (un vector) cuya dimensión, n , se preguntará por pantalla.

(1b) - Escribir un programa que cree un array bidimensional dinámico (una matriz) cuyas dimensiones $m \times n$ se preguntarán por pantalla.

(1c) – Escribir un programa que utilice arrays bidimensionales dinámicos para calcular el producto de dos matrices A y B . Las dimensiones de las matrices así como el valor de sus elementos se pedirán por pantalla. Se utilizarán tres funciones: una para introducir los elementos de cada matriz por pantalla; otra para realizar el producto de las matrices; y finalmente otra para mostrar en pantalla de forma ordenada la matriz producto.

$$A = \begin{pmatrix} 2 & 3 & 8 \\ 2 & 2 & 4 \\ 2 & 2 & 6 \end{pmatrix}; \quad B = \begin{pmatrix} 4 & 6 \\ 3 & 9 \\ 1 & 8 \end{pmatrix}$$

Apartado (1a):

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    cout<< "Introduce el valor de n: ";
    cin >> n;
    cout << endl;
    int *A= new int[n];
    // rellenamos el vector
    for(int i=0;i < n;i++){
        A[i]=(i*i+1);
        cout << A[i] << endl;
    }
    delete [] A;
    return 0;
}
```

Apartado (1b):

```
#include <iostream>
using namespace std;

int main()
{
double **A;
int fa, ca;
cout << "Introduce numero de filas: ";
cin >> fa;
cout << endl;
cout<< "Introduce numero de columnas: ";
cin >> ca;
cout << endl;
// reserva dinamica de memoria
A=new double* [fa];
for(int i=0;i < fa; i++) {
    A[i]=new double[ca];
}
// rellenamos la matriz
for(int i=0;i < fa; i++){
    for(int j=0;j < ca; j++){
        A[i][j]=2*i+3*j;
    }
}
// Se muestra la matriz
for(int i=0;i < fa; i++){
    for(int j=0;j < ca; j++){
        cout << A[i][j] << " ";
    }
    cout << endl;
}
// libera memoria
for (int i=0; i < fa; i++) {
    delete[] A[i];
}
delete [] A;
return 0;
}
```

Apartado (1c):

```
#include <iostream>
using namespace std;
void multmat(double **A, double **B, double **C, int fa, int ca, int cb);
void leermatriz (double **M, int f, int c);
void muestramatriz(double **M, int f, int c);
void borrararmatriz(double **M, int f);

int main()
{
    int fa, fb, ca, cb;
    double **A, **B, **C;
    cout << " introduzca numero de filas de A : ";
        cin >> fa;
    cout << " introduzca numero de columnas de A : ";
        cin >> ca;
    cout << " introduzca numero de filas de B : ";
        cin >> fb;
    cout << " introduzca numero de columnas de B : ";
        cin >> cb;
    int fc=fa, cc=cb;
    A = new double * [fa];
        for (int i = 0; i < fa; i++)
            A[i] = new double[ca];
    B = new double * [fb];
        for (int i = 0; i < fb; i++)
            B[i] = new double[cb];
    C = new double * [fc];
        for (int i = 0; i < fc; i++)
            C[i] = new double[cc];
    cout << "Introduce datos matriz A" << endl;
    cout << fa << "filas    y " << ca << "    columnas" << endl;
    leermatriz (A, fa, ca);
    muestramatriz(A,fa,ca);
    cout << "Introduce datos matriz B" << endl;
    cout << fb << "filas    y " << cb << "    columnas" << endl;
    leermatriz (B, fb, cb);
    muestramatriz(B,fb,cb);
    cout << endl;
    cout << "Matriz C:" << endl;
    multmat(A,B,C,fa,ca,cb);
    muestramatriz(C,fc,cc);
    // liberamos la memoria ocupada por A, B y C
    borrararmatriz(A,fa);
    borrararmatriz(B,fb);
    borrararmatriz(C,fc);

    return 0;
}
void multmat(double **A, double **B, double **C, int fa, int ca, int cb)
{
    double suma;
    for (int i=0; i<fa; i++) {
        for (int j=0; j<cb; j++) {
            suma=0.0;
            for (int k=0; k<ca; k++) {
                suma=suma+A[i][k]*B[k][j];
            }
            C[i][j]=suma;
        }
    }
}
void leermatriz (double **M, int f, int c)
{
    int i, j;
```

```

        for( i = 0; i < f; i++){
            for (j = 0; j < c; j++){
                cout << "INTRODUCE ELEMENTO ["<<i<<"]"<<["<<j<<"]"<< endl;
                cin >> M[i][j] ;
            }
        }
    }
}

void muestramatriz( double **M, int f, int c)
{
    for (int i=0; i<f; i++) {
        for (int j=0; j<c; j++) {
            cout << M[i][j] << " ";
        }
        cout << endl;
    }
}

void borrararmatriz(double **M, int f)
// libera memoria
{
    for (int i=0; i < f; i++) {
        delete[] M[i];
    }
    delete [] M;
}

```

Otra solución para el apartado 1(c) en las que se incluye una función adicional (crearmatriz) para reservar memoria dinámica para una matriz. Una función sólo puede devolver un valor. Si queremos tener como argumento de retorno una matriz, hay que devolver un puntero doble, es decir la dirección de memoria donde comienza la matriz. Nótese en este ejemplo como la función multmat tiene un puntero doble como valor de retorno:

```

#include <iostream>
using namespace std;
double** crearmatriz(double **M, int f, int c);
double** multmat(double **A, double **B, int fa, int ca, int cb);
void leermatriz (double **M, int f, int c);
void muestramatriz(double **M, int f, int c);
void borrararmatriz(double **M, int f);

int main()
{
    int fa, fb, ca, cb;
    double **A, **B, **C;
    cout << " introduzca numero de filas de A : ";
    cin >> fa;
    cout << " introduzca numero de columnas de A : ";
    cin >> ca;
    cout << " introduzca numero de filas de B : ";
    cin >> fb;
    cout << " introduzca numero de columnas de B : ";
    cin >> cb;
    int fc=fa, cc=cb;
    A=crearmatriz(A,fa,ca);
    B=crearmatriz(B,fb,cb);
    C=crearmatriz(C,fc,cc);
    cout << "Introduce datos matriz A" << endl;
    cout << fa << "filas   y " << ca << "   columnas" << endl;
    leermatriz (A, fa, ca);
    muestramatriz(A,fa,ca);
    cout << "Introduce datos matriz B" << endl;
    cout << fb << "filas   y " << cb << "   columnas" << endl;
}

```

```

leermatriz (B, fb, cb);
muestramatriz(B,fb,cb);
cout << endl;
cout << "Matriz C:" << endl;
C=multmat(A,B,fa,ca,cb); // La matriz C no se pasa como argumento
muestramatriz(C,fc,cc);
// liberamos la memoria ocupada por A, B y C
borrarmatriz(A,fa);
borrarmatriz(B,fb);
borrarmatriz(C,fc);
return 0;
}

double** crearmatriz(double **M, int f, int c)
{
    M = new double * [f];
    for (int i = 0; i < f; i++) { M[i] = new double[c];}
    return M;
}

void leermatriz (double **M, int f, int c)
{
    int i, j;
    for( i = 0; i < f; i++){
        for (j = 0; j < c; j++){
            cout << "INTRODUCE ELEMENTO ["<<i<<" "<<"["<<j<<" "<< endl;
            cin >> M[i][j] ;
        }
    }
double** multmat(double **A, double **B, int fa, int ca, int cb)
{
    double suma;
    double **Producto;
// al no pasar la matriz C hay que declarar la matriz Producto en la función
    Producto=crearmatriz(Producto,fa,cb);
    for (int i=0; i<fa; i++) {
        for (int j=0; j<cb; j++) {
            suma=0.0;
            for (int k=0; k<ca; k++) {
                suma=suma+A[i][k]*B[k][j];
            }
            Producto[i][j]=suma;
        }
    }
    return Producto;
}

void muestramatriz( double **M, int f, int c)
{
    for (int i=0; i<f; i++) {
        for (int j=0; j<c; j++) {
            cout << M[i][j] << " ";
        }
        cout << endl;
    }
}

void borrarmatriz(double **M, int f)
// libera memoria
{
    for (int i=0; i < f; i++) {
        delete[] M[i];
    }
    delete [] M;
}

```

(2) - Escribir un programa que inicialice y muestre por pantalla un array de 6 elementos utilizando punteros.

```
// inicializacion y visualizacion de un array con punteros
#include <iostream>
using namespace std;

int main()
{
    double V[6];
    for (int j = 0; j < 6; j++)
        *(V+j) = (j + 1) * 10 + 1;
    cout << "    Direccion    Contenido" << endl;
    for (int j= 0; j < 6; j++)
    {
        cout << "  V+" << j << " = " << V+j ;
        cout << "      V[" << j<<"] = "<< *(V+j)<< "\n" ;
    }

    return 0;
}
```